

packages/graph-core/src/types.test.ts

Kind: added · Baseline: a9c34bfb03fe · Target: NovaDiff · Generated: 2026-05-20T05:28:56.090Z

Overview

A new test suite `packages/graph-core/src/types.test.ts` has been added to validate the public type contracts of the KnowledgeGraph module. It exercises node/edge creation, type aliases, optional fields, and backward-compatibility guarantees.

Key changes

- **Imports:** Added `vitest` test helpers and type imports from `./types.js`.
- **KnowledgeGraph construction:** Tests create a minimal graph instance and assert all required properties (`version` , `project` , `nodes` , `edges` , `layers` , `tour`).
- **GraphNode & GraphEdge:** Dedicated tests confirm that all mandatory fields are present and optional fields (`filePath` , `lineRange` , `languageNotes`) behave correctly for different node/edge kinds.
- **Type aliases:** `NodeType` and `EdgeType` arrays are fully enumerated (13 node types, 26 edge types) and verified against the corresponding union types.
- **StructuralAnalysis:** Tests ensure optional non-code fields (`sections` , `definitions` , `services` , etc.) are correctly handled and that backward-compatibility (missing fields) is preserved.
- **AnalyzerPlugin:** Optional `resolveImports` and `extractReferences` callbacks are exercised, confirming that missing callbacks default to `undefined` and that `extractReferences` returns the expected array.

Impact

- **Correctness:** Guarantees that the public API remains type-safe and that future changes to the type definitions will be caught early.
- **Maintainability:** Centralized tests for type contracts reduce the risk of accidental breaking changes when refactoring the graph model.
- **Compatibility:** Explicitly tests backward-compatibility for optional fields, ensuring older consumers are not affected by new optional properties.
- **Observability:** Test failures will surface immediately if any type contract is violated, providing clear diagnostics.

Risks & follow-ups

- **Test coverage:** The suite covers only the happy path; edge cases such as malformed input objects are not tested. Consider adding negative tests.
- **Future type changes:** Adding new node or edge kinds will require updating the corresponding arrays; missing updates could cause false positives.

- **Performance:** The tests are lightweight, but running them in CI may increase build time marginally; monitor if the suite grows.
- **Documentation sync:** Ensure that the type definitions in `./types.js` remain in sync with the test expectations; otherwise, tests may fail spuriously.